

Projet Python — Groupe 5

Fonctions, générateurs, tri, classes, pandas et polars

Enseignante : Rahma GARGOURI — 2026-2027

Présentation finale — Projet en groupe

Vous préparez une **présentation** (PowerPoint, PDF ou autre format) qui répond aux questions ci-dessous, accompagnée d'un **notebook Jupyter** contenant les solutions des exercices, à exécuter pendant la présentation.

Partie A — Fonctions, générateurs et algorithmes de tri

Notion nouvelle (non vue en séance) : les générateurs. Une fonction qui utilise `return` calcule tout son résultat, le renvoie, puis s'arrête. Une fonction qui contient `yield` devient un **générateur** : chaque fois qu'une valeur est demandée (par exemple par une boucle `for`), elle s'exécute jusqu'au prochain `yield`, fournit cette valeur, puis se met en pause jusqu'à la demande suivante.

```
def compter_jusqua(n):
    for i in range(1, n + 1):
        yield i

for x in compter_jusqua(3):
    print(x)      # affiche 1, puis 2, puis 3
```

Questions

1. Qu'est-ce qu'une fonction `lambda` ?
2. Quelle est la différence entre `return` et `yield` dans une fonction ? Qu'est-ce qu'un générateur ?
3. Pourquoi un générateur utilise-t-il très peu de mémoire ? Que se passe-t-il si on le parcourt deux fois ?
4. Quelle est la complexité de chacun des trois algorithmes de tri de l'exercice A.2 ? Lequel est le plus rapide, et dans quels cas ?

Exercices

A.1 — `return` contre `yield`. Écrire `carres_liste(n)`, qui renvoie avec `return` la liste des carrés de 0 à $n - 1$, et `carres_generateur(n)`, qui produit les mêmes valeurs avec `yield`. Pour $n = 1\,000\,000$, comparer la mémoire occupée par les deux résultats (`sys.getsizeof`), puis calculer deux fois de suite `sum(...)` sur le générateur et expliquer le résultat.

A.2 — Trois algorithmes de tri. Écrire, sans utiliser `sorted`, trois fonctions qui trient une liste d'entiers :

- **tri par sélection** : on cherche le plus petit élément, on le place en premier, puis on recommence sur le reste de la liste ;
- **tri fusion** : on coupe la liste en deux, on trie chaque moitié (récursivement), puis on fusionne les deux moitiés triées en une seule liste triée ;

- **tri par comptage** : pour des entiers compris entre 0 et k , on compte combien de fois chaque valeur apparaît, puis on reconstruit la liste dans l'ordre.

Mesurer leur temps d'exécution (`time.perf_counter`) sur des listes de n entiers aléatoires entre 0 et 1 000, pour $n = 1\,000, 2\,000, 4\,000$ et $8\,000$, et comparer avec `sorted`. Que devient le temps de chaque tri quand n double ?

Partie B — Classes

Notion nouvelle (non vue en séance) : l'héritage. Une classe **fil** peut **hériter** d'une classe **mère** : elle récupère automatiquement tous ses attributs et toutes ses méthodes, et peut en ajouter d'autres. On l'écrit `class Fille(Mere):`. Dans le `__init__` de la classe fille, `super().__init__(...)` appelle le `__init__` de la classe mère, pour ne pas réécrire son code.

```
class Animal:
    def __init__(self, nom):
        self.nom = nom
    def presenter(self):
        return f"Je m'appelle {self.nom}"

class Chien(Animal):          # Chien hérite de Animal
    def __init__(self, nom, race):
        super().__init__(nom) # appelle __init__ de Animal
        self.race = race      # attribut ajouté par Chien

Chien("Rex", "labrador").presenter() # "Je m'appelle Rex" (méthode héritée)
```

Questions

1. Qu'apporte une `dataclass` par rapport à une classe écrite à la main ?
2. Qu'est-ce que l'héritage ? À quoi sert `super()` ?
3. À quoi sert `self` ?

Exercices

B.1 — Transactions. Définir une `dataclass Transaction` (`date`, `devise`, `montant`), convertir la liste de dictionnaires ci-dessous en liste de `Transaction`, puis écrire `total_par_devise`.

```
transactions = [
    {"date": "2026-01-05", "devise": "USD", "montant": 120.0},
    {"date": "2026-01-06", "devise": "GBP", "montant": 80.0},
    {"date": "2026-01-07", "devise": "USD", "montant": 300.0},
    {"date": "2026-01-08", "devise": "JPY", "montant": 50.0},
    {"date": "2026-01-09", "devise": "GBP", "montant": 210.0},
]
total_par_devise(objets) # objets = la liste de Transaction
# {'USD': 420.0, 'GBP': 290.0, 'JPY': 50.0}
```

B.2 — Héritage. Écrire une classe `Compte` (attributs `titulaire` et `solde`; méthode `deposer(montant)`). Écrire une classe `CompteEpargne` qui hérite de `Compte`, ajoute un attribut `taux` (en initialisant le reste avec `super().__init__(...)`) et une méthode `ajouter_interets()` qui applique $\text{solde} \leftarrow \text{solde} \times (1 + \text{taux})$. Montrer qu'un `CompteEpargne` peut utiliser `deposer` sans la réécrire.

Partie C — pandas et polars

Installer `polars` avec `pip install polars`.

Questions

1. Quelles sont les différences entre `pandas` et `polars` (langage dans lequel ils sont écrits, utilisation de plusieurs cœurs du processeur, évaluation paresseuse, syntaxe) ?
2. Comment écrire une moyenne mobile sur 5 jours en `pandas`, et en `polars` ?

Exercices

C.1 — pandas contre polars. Créer un tableau de 10 millions de lignes (colonne `groupe` : entier aléatoire de 0 à 999 ; colonne `valeur` : nombre aléatoire) et l'enregistrer en CSV. Mesurer ensuite avec `time.perf_counter`, en `pandas` et en `polars` :

- la lecture du CSV ;
- la moyenne de `valeur` par `groupe` ;
- la moyenne mobile de `valeur` sur 5 lignes.

Présenter un tableau des temps. Afficher le nombre de cœurs de votre machine (`os.cpu_count()`) et expliquer les écarts observés.